Specification of team software project

Department of Software Engineering

Faculty of Mathematics and Physics, Charles University

Solvers: Vojtěch Kloda

Study program: Computer Science - Software and Data Engineering

Project title: The influence of localization on video search engine performance

Project type: Research project

Supervisor: doc. RNDr. Jakub Lokoč, Ph.D.

Consultant: Bastian Jäckl, M.Sc.

1 Introduction

This project will be conducted in collaboration with the team developing the PraK tool; mainly with Jakub Lokoč, who will be the project supervisor, and secondarily Bastian Jäckl, the project consultant. The team is a yearly participant of the Video Browser Showdown (VBS), an annual live video search competition for researchers, in which I, as a member of the PraK team, also participated. The team uses its PraK tool for the competition and actively searches for new innovations for the next year. During a pre-study I conducted, I found that using localization leads to better search results. The rest of the team was interested in these findings and asked me to conduct a research project on the effectiveness of localization, so that it can be exploited in the tool.

My project will be to collect data and develop tools for analysing the influence of localization in text-to-image video search to assess the feasibility of using localization in a live setting.

1.1 Relation to other projects

The project will be related to the PraK tool, as it aims to improve it. A localization method I developed from the limited findings of the pre-study was already integrated into the PraK tool and debuted in this year's competition.

This project will also be related to the research on texture embeddings conducted by Bastian Jäckl. His project will assess the viability of replacing CLIP embedding used by the PraK tool currently with embeddings derived from image textures. Our goal is to combine both studies into one paper submitted to the SISAP conference.

In the scope of this project, I will cooperate with Jakub Lokoč, who will guide me and provide iterative feedback to my findings, and Bastian Jäckl, who would help me with running GPU intensive tasks on the school's *gpulab* in case it would be necessary.

Outside of this project, I will cooperate with other members of the PraK team on how to integrate the findings into the tool.

2 Project description

The goal of the project is to test the limits of localizations on a well-known dataset and document the results.

Localization is a technique for limiting the context passed to text-to-image models to improve performance, usually implemented with grid search. This approach splits the dataset into grid cells and allows for querying specific cells. A simple example would be to split the dataset into four corner cells. If a user wanted to find an object in the upper right part of a frame, he could select the upper right corner cell and execute the query inside it. The system would pass the localized part of the dataset to the text-to-image model and rank its frames according to the similarity to the query. The idea is that the text-to-image model performs better when most of the original image is left out, leaving out context that might not be related to what the user is looking for.

This project will compare three grid systems. The first one will be the hypothetically optimal grid, a grid that contains all possible cells, including overlapping ones. Thus, a user could select the best possible rectangular area of the dataset for his queries. Such a system cannot be used in practice because it would require precomputing embeddings for all subimages for every frame of the dataset. However, this system can be simulated. The performance increase of such a system would become the upper ceiling for grid search.

The second grid system will be the static grid. Instead of having every region precomputed, the static grid only has a select few static regions available, just like the corner grid example at the beginning of the chapter. The project will analyze the best performing static grid found during the pre-study. It is similar to the corner grid from the example, but has an added overlapping central cell and increased side lengths depicted in figure 1. This was also the localization method used in the PraK tool for this year's VBS competition.

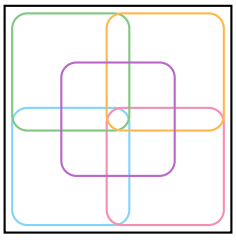

Figure 1: Static grid used in the PraK tool.

The third and final grid system will be the dynamic grid. Similarly to the optimal grid, it allows the user to select any image region for query execution. However, it does not precompute the embeddings for all regions, but only for those deemed the most important.

To determine region importance, object extraction methods can be used to find regions containing objects of interest, such as animals and divers, in the dataset frames. When the user queries a specific region, the system selects the most relevant precomputed region for each frame.

The three grid systems will be evaluated using data provided by volunteers, simulating inputs for a video search system using these grids. The dataset selected for this study is the Marine Video Kit (MVK) because it is used in the VBS competitions and because its frames were already extracted in a different project by the PraK team.

2.1 Functional requirements

2.1.1 Annotation GUI

The Annotation GUI will be a standalone desktop application that will allow volunteers to view frames of the MVK dataset and annotate them with hand-drawn rectangles and text. These annotations will simulate user inputs of a video search system in a live setting.

1. [GUI-1] Dataset viewing: The application displays frames from the MVK dataset to the user.
2. [GUI-2] Dataset sampling: The displayed frames are sampled randomly to limit the number of duplicit annotations.
3. [GUI-3] Dataset annotation: The application allows the user to draw a rectangle on the displayed frame and add a textual description relevant to the area inside the rectangle.
4. [GUI-4] Annotation submission: The application allows the user to submit the annotation. The sumbitted annotations will be aggregated into a single local JSON file, containing the ID of the image, coordinates of the bounding box, and the textual annotation.

2.1.2 Processing Tool

The Processing Tool ingests user annotations and provides an interface for the other tools to access the annotations, the MVK dataset frames, and various text-to-image models for evaluation.

1. [PROC-1] Annotation interface: There shall be an interface for the optimal, static, and dynamic analysis tools to retrieve user annotations.
2. [PROC-2] Model interface: There shall be an interface that yields the relevant Open CLIP model objects for text and image feature extraction for the models used in the analysis tools.
3. [PROC-3] Dataset interface: There shall be an interface that loads and returns the images from the MVK dataset, including relevant metadata, such as identifiers and filepaths.

2.1.3 Optimal Grid Analysis Tool

The Optimal Grid Analysis Tool can simulate the hypothetically optimal grid system introduced in the project description and analyze its performance.

Each produced MVK annotation will serve as a query for the system, with the annotated object's bounding box serving as the selected grid cell, and the annotation text as the query. The tool will simulate each query by creating a new, derived MVK dataset cropped to the coordinates of the object bounding box and extracting its frame embeddings (called cell embeddings, where each cell embedding represents a single frame). It will then use a text-to-image model to rank the cell embeddings by similarity to the query, finding out how high the cell embedding representing the annotated frame is ranked. This will yield the same results as if the embeddings for all cells were precomputed.

Additionally, the tool will also measure how the grid system performs with more added context (greater image area around the annotated object). This can be done by repeating the process for the same query but increasing the size of the bounding box around the annotated frame object.

1. [OPTIMAL-1] Cell embedding extraction: The tool is capable of creating new, derived datasets from the original MVK dataset that are cropped to a specified bounding box and extract their cell embeddings. These bounding boxes are retrieved from the user annotations.

2. [OPTIMAL-2] Similarity measurement: The tool is capable of performing a text-to-image query for the given query and cell, compute the resulting similarity rankings of cell embeddings, and track each cell embedding back to the original frame. The annotation texts will be used as queries.

3. [OPTIMAL-3] Annotation processing: The tool can create cell embeddings for every user annotation, as well as cell embeddings with added context created by enlarging the bounding box by a given algorithm. The tool can then execute the annotation text as a query in the cell and enlarged cells and compute the rankings of the annotated frame.

4. [OPTIMAL-4] Statistical dataset generation: The tool is capable of producing a statistical dataset for a given text-to-image model that contains the following information:

   (a) The identifier of the annotation.

   (b) The rank of the annotated frame in the original MVK dataset (no localization).

   (c) The rank of the annotated frame in all cells created for the annotation.

   (d) The ratio of the cell area compared to the annotation bounding box for all cells created for the annotation.

2.1.4 Static Analysis Tool

The Static Analysis Tool measures the performance of the static grid used in the PraK tool (depicted in figure 1).

The tool will create derived datasets and extract their embeddings for the five cells of the static grid and execute annotation texts as queries in the cell with the greatest intersection-over-union with the annotation bounding box. The intersection-over-union cell selection simulates the user choosing the most suitable cell for the query.

1. [STATIC-1] Static cell embedding extraction: The tool can extract the cell embeddings for the five cells of the static grid.

2. [STATIC-2] Object selection: For each annotation bounding box, the tool can select the cell with the greatest intersection-over-union.

3. [STATIC-3] Annotation processing: The tool can execute annotation texts as queries in the selected cell, yielding the rank of the annotated frame.

4. [STATIC-4] Statistical dataset generation: The tool can produce a statistical dataset for a given text-to-image model that contains the following information:

(a) The identifier of the annotation.

(b) The rank of the annotated frame in the cell with the greatest intersection-over-union.

2.1.5 Dynamic Analysis Tool

The Dynamic Analysis Tool measures the performance of the dynamic grid.

The tool will find objects in the MVK dataset using an object extraction method and extract the embeddings of their bounding boxes. The annotation bounding boxes will be used as the query cells for which the tool will select the embedding with the greatest intersection-over-union for each frame. These embeddings will be ranked by similarity to the query like in the other two tools.

1. [DYNAMIC-1] Object extraction: The tool can find object bounding boxes in the MVK dataset and extract their embeddings.

2. [DYNAMIC-2] Object selection: For each annotation bounding box, the tool can use the intersection-over-union metric to select the most suitable embedding for each frame.

3. [DYNAMIC-3] Annotation processing: The tool can use the annotation texts as queries and compute the similarity ranking with the embeddings selected in DYNAMIC-2.

4. [DYNAMIC-4] Statistical dataset generation: The tool is capable of producing a statistical dataset for a given model that contains the following information:

   (a) The identifier of the annotation.

   (b) The rank of the annotated frame when querying the system as described in DYNAMIC-3.

   (c) The area of the annotation bounding box.

2.2 Non-Functional requirements

1. [NF-1]: Every grid analysis should be run on at least two distinct image-to-text models. The selected models will be the ones used by the PraK tool on VBS competitions to see whether there is a trend of increased or decreased localization performance over the years.

2. [NF-2]: The number of cells per annotation in the optimal grid analysis should be at least two. This is to see whether adding some extra context to the localization method increases its performance.

2.3 Mockups

There will be only one screen, as the rest of the software will be CLI tools. The Annotation GUI allows users to view dataset pictures, draw rectangles around objects of interest, and add textual annotations.

Below is a screenshot of a demo version of the Annotation GUI, showcasing all of the required features, as well as displaying additional metadata, such as the file path and selection coordinates.

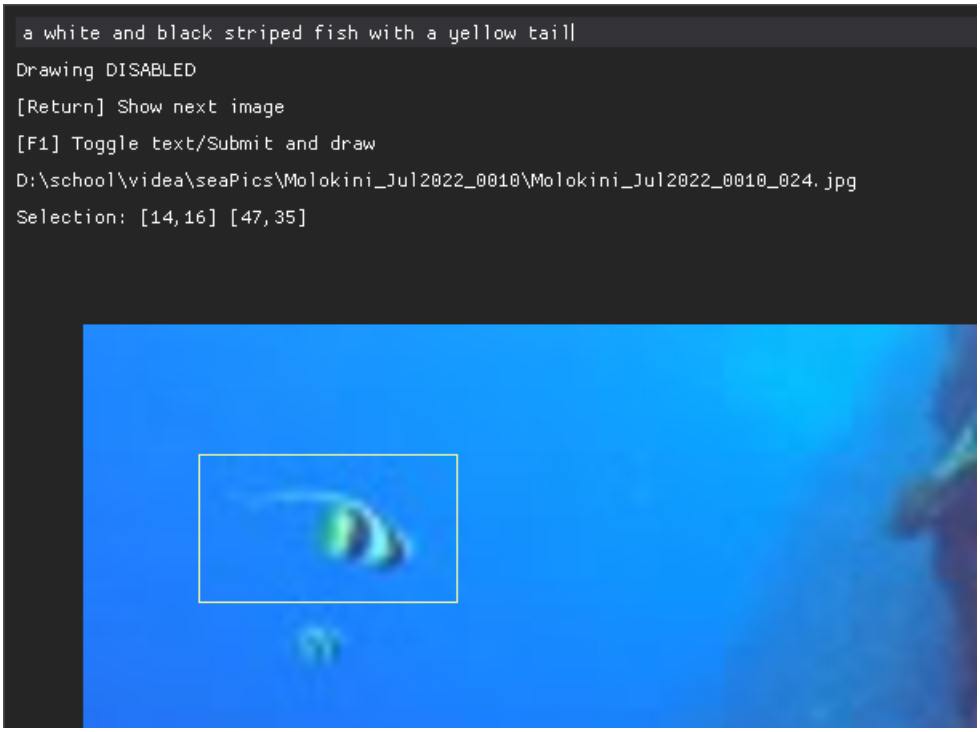

Figure 2: The Annotation GUI demo, with a hand-drawn rectangle in the middle and a description on the top.

2.4 Architecture

The project will be divided into three parts:

- Annotation GUI: Produces annotation files in JSON format.
- Database: Stores annotation files, MVK frames, and the embeddings produced by the tools.
- Processing and analysis tools: Create embeddings from frames and use annotations for performance evaluation.

2.4.1 Annotation GUI

The Annotation GUI produces a JSON file for each user, which will be sent to me. Because the expected number of volunteers annotating the files is around 10, it was reasoned that there is no need for a sophisticated submission system, thus sending the files by email will be sufficient.

2.4.2 Database

The database will store the annotation files, MVK frames, and embeddings. The annotations and frames will be stored permanently, whereas most of the embeddings will be single-use. Both the static and dynamic analysis will produce a single embedding set, and the optimal grid analysis will produce a set for each annotation, which can be deleted after the annotation was processed.

The database will be realized with a folder structure on my local file system and possibly duplicated on *gpulab*.

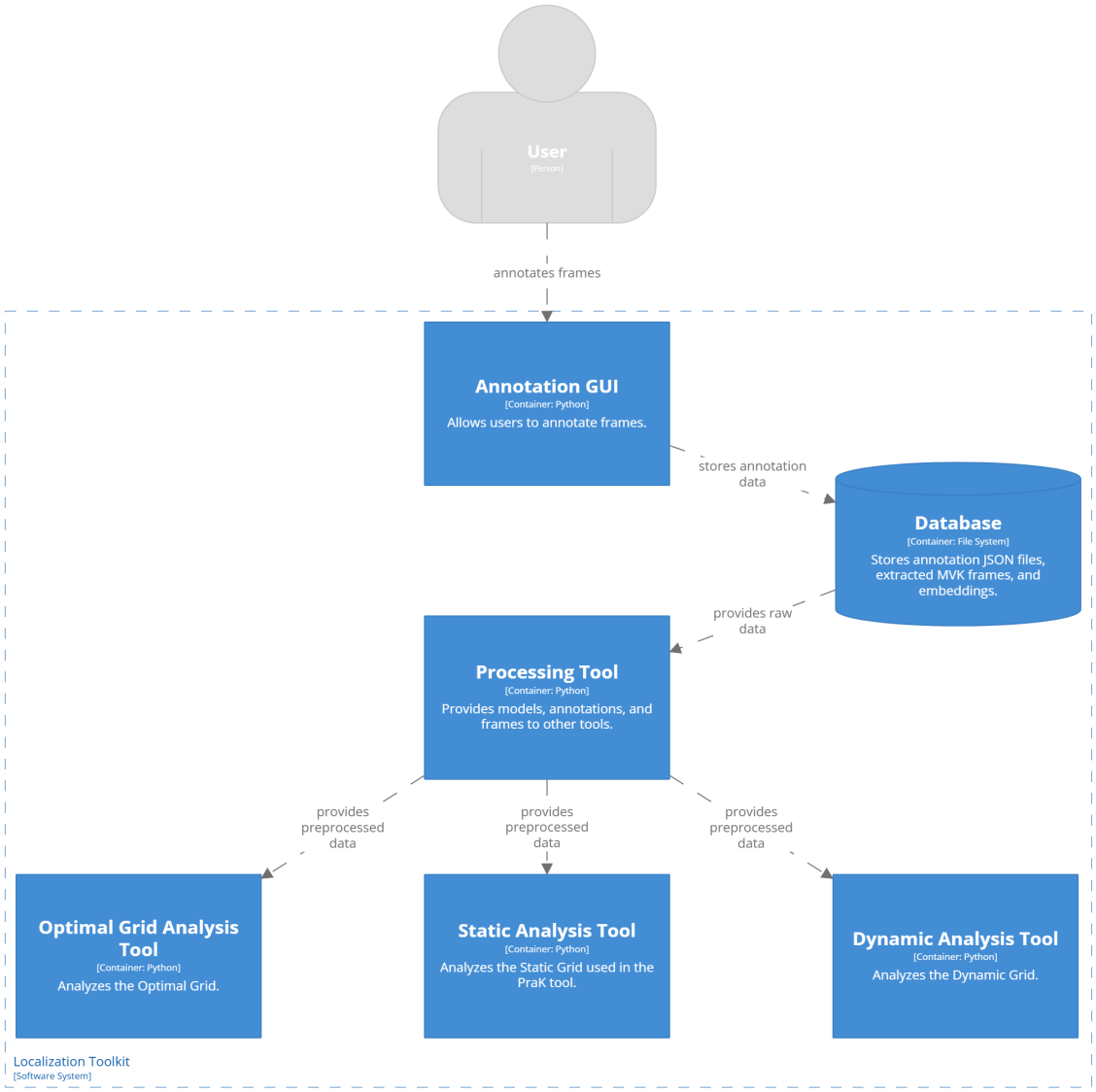

Figure 3: The architecture of the localization toolkit.

The reason for this decision is two-fold. The first reason is that the MVK frames are already organized in a folder structure on *gpulab* due to various PraK-related projects, as well as on my machine.

The second reason is that both the frames and embeddings would have to be stored as blobs, with close to no reason to use sophisticated querying methods on them. Extracting embeddings from frames will always be done in a sequential order, and created embeddings will only be used once in the optimal grid analysis.

The only hindrance would be to move the annotations and some embeddings from my local machine to *gpulab* in case more computing power is needed, but that would only mean moving a few files.

2.4.3 Processing and Analysis Tools

The Processing Tool exposes a simple API used by the three analysis tools running on the same machine. Thus, it was reasoned that making it a standalone application would not provide any benefits compared to making it a

library that can be included by the other tools, making the system simpler.

3 Platform and technologies

The Annotation GUI will have minimal hardware requirements so that it can be run on the local machines of the volunteers. The tool will target Windows platforms.

The analysis tools will run on the CLI because a GUI would not add much value to them. They will be computationally intensive because they require many image embeddings to be precomputed, and later on compared to text embeddings. Thus, the tools will be implemented so that they can delegate as much of the work to GPUs as possible. Initially, these tools will be run on my personal computer fitted with an NVIDIA GeForce RTX 3070 GPU, with 32 GB of DDR5 RAM (4800 Hz), and an Intel 13700K processor. In case more computing power will be needed, the tools will be run on the school's *gpulab*. The tools will support both Windows and Linux platforms, because the WSL2 environment on my computer has trouble using the GPU effectively, and *gpulab* uses Linux.

All of the software will be written in Python, as it has great support for deep learning related tasks. The processing tools will rely on the following commonly used libraries:

- *torch*: tensor calculations and manipulation (on GPUs),
- *open_clip*: text-to-image search framework,
- *pickle*: Python object serializer and parser,
- *PIL*: image manipulation,
- *cv2*: computer vision and object extraction (this library could be replaced by a more suitable one in the final iteration).

The annotation GUI will use *dearpygui*, a powerful GUI framework for Python.

4 Risks

4.1 Lack of Data

The greatest risk is that volunteers will not provide enough data. A lack of at least several hundred annotations could degrade the statistical output of the project. This risk can be partly mitigated by providing the rest of the annotations myself, but that will come at the cost of biasing the annotations towards my writing style. A diverse collection of annotations by many people would better simulate a system used by different users.

4.2 Performance

Another risk is performance. This project will be computationally heavy, as each annotation will produce new derived datasets and cell embeddings in the optimal grid analysis, and the dynamic analysis requires extracting objects from the whole dataset. If my personal computer cannot suffice, the school's *gpulab* will have to be used to

speed up the computation. If even *gpulab* would take too long, the scope of the analysis will need to be reduced, meaning fewer cells per annotation in the optimal grid analysis and a greater pruning of extracted objects in the dynamic analysis.

5 Milestones / Deliverables

The main outcome of the project are graphs that show the performance increases of the three grid systems analyzed. These will be cumulative graphs showing how many annotated images ranked inside a threshold represented by the X axis of the graphs. Such graphs are the go-to option for performance comparisons of video search engines, usually plotting the cumulative distribution functions for different text-to-image models or new technologies, such as localization.

These graphs will be presented in a report detailing the specific methods used throughout the project. They will compare the performance of the selected text-to-image models for all analyzed grid systems and compare the grid systems with each other.

The tools and GUI used in the project will be available on GitHub, but only the GUI can be effectively presented to the committee, as the analytical CLI tools take a long time to run.

6 Time Schedule

The schedule below shows a conservative timeline, reserving enough time for frame annotation and processing. The greatest risks to this timeline are annotators taking longer than expected and complications during processing, especially during the optimal grid analysis.

It was assumed that the implementation of the tools themselves should be relatively straightforward and that no major complications should arise. The implementation can take place during processing, enabling parallelization.

Milestone / Deliverable	Activity	Time schedule	Man-days
Annotation GUI	Implementation	Month 1	8
	Testing	Month 1	4
Data gathering	Sourcing people for annotations	Month 1	6
	Waiting for annotators to create the data	Month 2	20
Processing tool	Tool implementation	Month 2	2
	Tool testing	Month 2	1
	Tool documentation	Month 2	1
Optimal grid analysis	Tool implementation	Month 2	8
	Tool testing	Month 2	2
	Tool documentation	Month 2	1
	Running the analysis	Month 3	20
Static analysis	Tool implementation	Month 2	3
	Tool testing	Month 2	1
	Tool documentation	Month 2	1
	Running the analysis	Month 3	3
Dynamic analysis	Tool implementation	Month 3	8
	Tool testing	Month 3	2
	Tool documentation	Month 3	1
	Running the analysis	Month 4	10
Evaluation	Optimal grid analysis evaluation	Month 5	2
	Static analysis evaluation	Month 5	2
	Dynamic analysis evaluation	Month 5	2
	Comparative evaluation	Month 5	2

7 Form of collaboration

7.1 Consulting plan

Me and other members of the PraK team (including my supervisor) meet every week to discuss progress on the tool and individual research projects. Traditionally, a participant is picked every week to make a presentation on their progress so that everyone can provide comments. After the presentation, participants can use the remainder of the meeting for individual or group consultations. These meetings usually take one to two hours.

Additional meetings can be organized, but they are rare due to the effectiveness of weekly meetings.

7.2 Collaboration with other team members

I will have regular progress consultations with my supervisor at team meetings, where I can ask him for guidance when needed.

I will work together with my consultant on a submission to the SISAP conference, in which we would present both of our projects. We decided to join our research projects in this way because both of them aim to improve the performance of text-to-image embeddings. Although my project focuses on improving CLIP embeddings using localization and his on replacing CLIP embeddings with texture embeddings, both of these methods can be used in video search engines together, as one method may outperform the other on one dataset but fall behind on another.

In addition, I will work with my consultant on how to efficiently run the analysis tools on *gpulab*.

Outside of the scope of this project, I will collaborate with other PraK members on how best to integrate the findings into the tool.